

Sistemas Operacionais

Memória Virtual

- Por Partições
- Por Segmentos
- Por Paginações



Windows



Mac OS



Linux



Chrome OS



Android



iOS



Unix



BSD

Hardware de Memória

A memória principal é um componente fundamental em qualquer sistema de computação. Constitui o “espaço de trabalho” do sistema, no qual são mantidos os processos, threads e bibliotecas compartilhadas, além do próprio núcleo do sistema operacional, com seu código e suas estruturas de dados.

O hardware de memória pode ser bastante complexo, envolvendo diversas estruturas, como memórias **RAM**, **caches**, **unidade de gerência**, **barramentos**, etc, o que exige um esforço de gerência significativo por parte do sistema operacional.

Hardware de Memória

A memória de um computador é na verdade uma grande tabela com células sequenciais, cada célula tem seu próprio endereço que segue um padrão contínuo.

Hardware de Memória

Código Fonte

```
#include <stdio.h>

int main() {
    char letra = 'A'; //0x01A
    int num = 1500; //0x01E
    double pi = 3.1415; //0x00F

    printf ("\n%d", sizeof(letra));
    printf ("\n%d", sizeof(num));
    printf ("\n%d", sizeof(pi));

    printf ("\n%d", letra);
    printf ("\n%c", letra);

    return 0;
}
```

Em Geral

inteiros = 4 bytes
float = 4 bytes
double = 8 bytes
char = 1 byte

Endereço	Memória
0x00F	0000000
0x010	0110001
0x011	0100001
0x012	11001010
0x013	11000000
0x014	10000011
0x015	00010010
0x016	01101111
0x017	
0x018	
0x019	
0x01A	01000001
0x01B	
0x01C	
0x01D	
0x01E	00000000
0x01F	00000000
0x020	00000101
0x021	11011100
0x022	
0x023	
0x024	
0x025	

Hardware de Memória

Podemos considerar,

a primeira célula será	0x00F, Conteúdo [00000000]
...a segunda	0x01F, Conteúdo [00000000]
...a terceira	0x02F, Conteúdo [00000000]
...a quarta	0x03F, Conteúdo [00000000]
...a quinta	0x04F, Conteúdo [00000000]
...a sexta	0x05F, Conteúdo [00000000]
...a sétima	0x06F, Conteúdo [00000000]
...a oitava	0x07F, Conteúdo [00000000]

Hardware de Memória

Existem diversos tipos de memória em um sistema de computação, cada um com suas próprias características e particularidades, mas todos com um mesmo objetivo: armazenar informação.



Hardware de Memória

Tipicamente é possível identificar vários locais onde dados são armazenados: **os registradores e o cache interno do processador (denominado *cache L1*), o cache externo da placa mãe (*cache L2*) e a memória principal (RAM).**

Além disso, discos e unidades de armazenamento externos (***pendrives*, CD-ROMs, DVD-ROMs, fitas magnéticas, SSDs, etc.**)

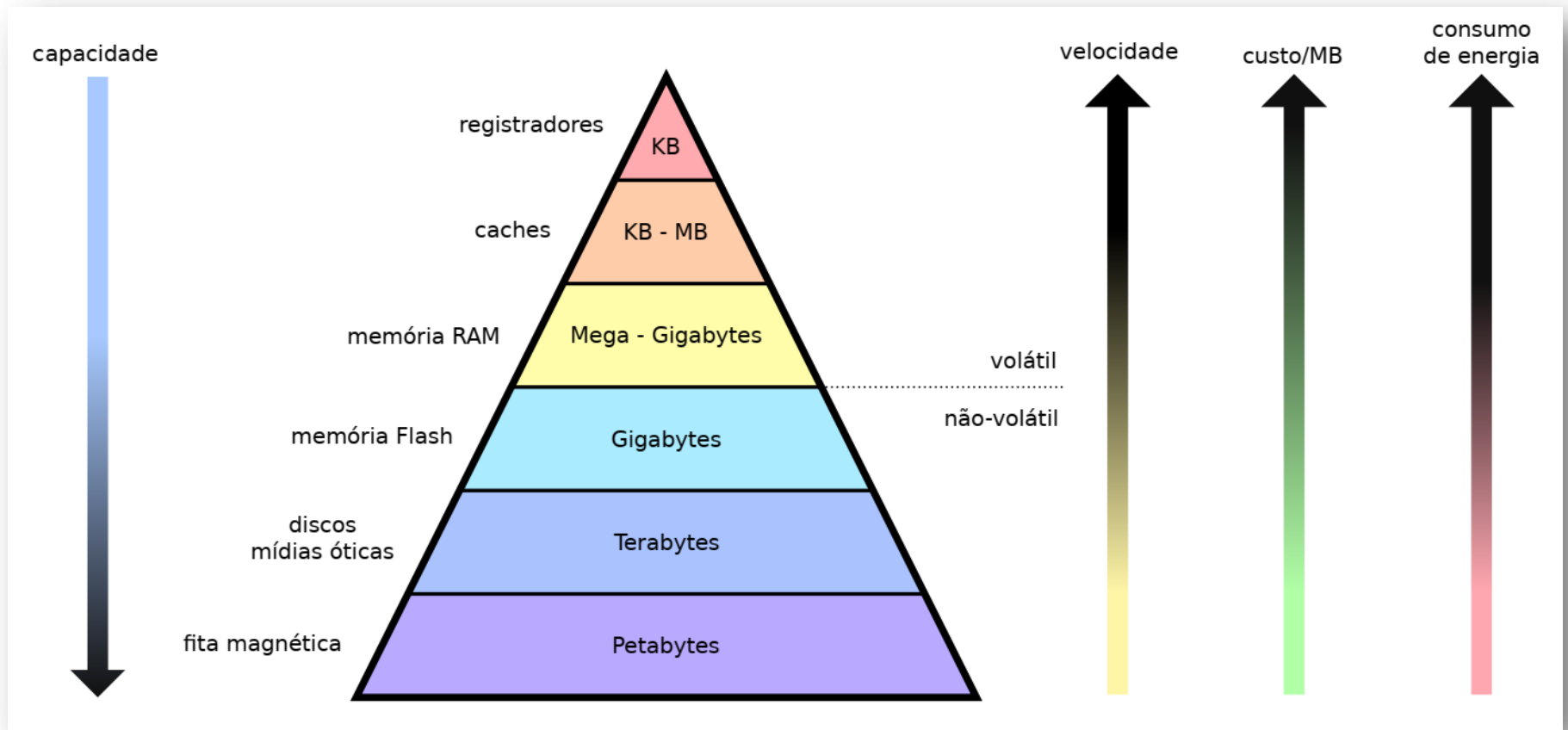
Hardware de Memória

Normalmente as memórias mais rápidas, como os registradores da CPU e os *caches*, têm menor capacidade de armazenamento, mais caras e consomem mais energia que memórias mais lentas, como a memória principal (RAM) e os discos.

Além disso, as memórias mais rápidas são *voláteis*, ou seja, perdem seu conteúdo ao ficarem sem energia, quando o computador é desligado.

Memórias que preservam seu conteúdo mesmo quando não tiverem energia, como as unidades *flash* e os discos rígidos, são denominadas *memórias não-voláteis*.

Hardware de Memória



Hardware de Memória

Outra característica importante das memórias é a **rapidez de seu funcionamento**, que pode ser traduzida em duas grandezas: o *tempo de acesso* (ou *latência*) e a *taxa de transferência*.

O tempo de acesso caracteriza o tempo necessário para iniciar uma transferência de dados de/para um determinado meio de armazenamento.

Por sua vez, **a taxa de transferência** indica quantos bytes por segundo podem ser lidos/escritos naquele meio, uma vez iniciada a transferência de dados .

Hardware de Memória

Meio	Tempo de acesso	Taxa de transferência
Cache L2	1 ns	1 GB/s (1 ns por byte)
Memória RAM	60 ns	1 GB/s (1 ns por byte)
Memória <i>flash</i> (NAND)	2 ms	10 MB/s (100 ns por byte)
Disco rígido SATA	5 ms (tempo para o ajuste da cabeça de leitura e a rotação do disco até o setor desejado)	100 MB/s (10 ns por byte)
DVD-ROM	de 100 ms a vários minutos (caso a gaveta do leitor esteja aberta ou o disco não esteja no leitor)	10 MB/s (100 ns por byte)

Tempos de acesso e taxas de transferência típicas [Patterson and Hennessy, 2005]

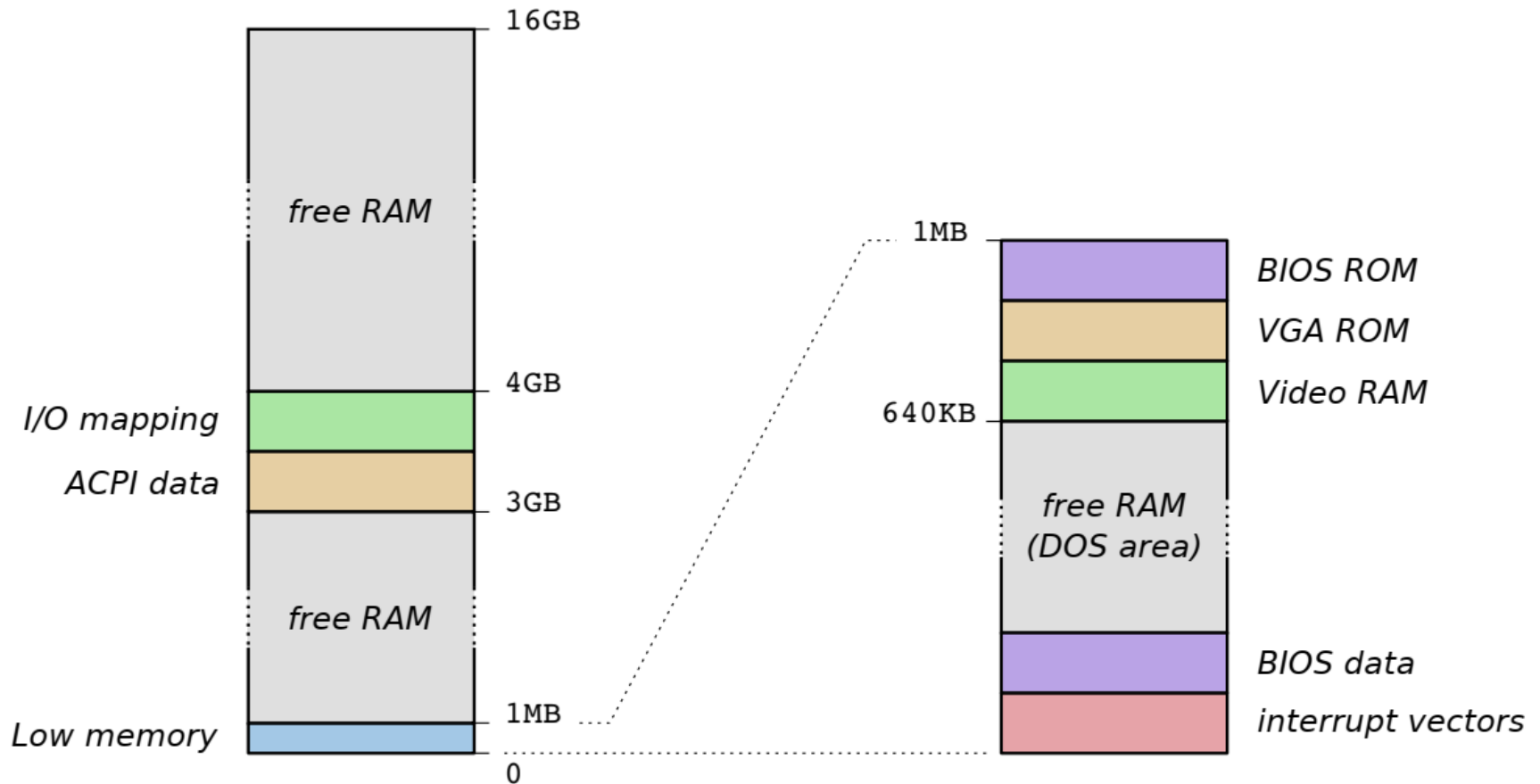
A memória Física

A memória principal do computador é uma área de RAM composta **por uma grande sequencia de bytes**, que é a menor unidade de memória usada pelo processador.

Cada byte da memória RAM possui um endereço, que é usado para acessá-lo.

A quantidade de memória **RAM** disponível em um computador constitui seu **espaço de memória física**.

A memória física



Organização (simplificada) da memória RAM de um computador atual com 16 GB de memória RAM.

A memória física

Os sistemas atuais mais sofisticados usam os conceitos de **espaços de endereçamento e de memória virtual**, para desacoplar a **visão da memória pelos processos da estrutura física da memória no hardware**.

Esse desacoplamento visa tornar mais simples e flexível o uso da memória pelos processos e pelo sistema operacional.

A memória física

Espaço de endereçamento

O processador acessa a memória RAM através de barramentos de dados, de endereços e de controle.

O barramento de endereços (como os demais) possui um número fixo de vias, que define a quantidade total de endereços de memória que podem ser gerados pelo processador: um barramento de dados com n vias consegue gerar 2^n endereços distintos.

Ex: Intel 80386 possui 32 vias de endereços $\rightarrow 2^{32} = 4\text{GB}$

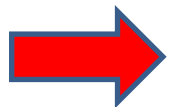
Intel Core i7 usa 48 vias para endereços $\rightarrow 2^{48} = 256\text{TB}$

A memória Virtual

Para ocultar a organização complexa da memória física e simplificar os procedimentos, é implementado:

Endereços físicos (ou reais) são os endereços dos bytes de memória física do computador. Estes endereços são definidos pela quantidade de memória disponível na máquina

Endereços lógicos (ou virtuais) são os endereços de memória usados pelos processos e pelo sistema operacional e, portanto, usados pelo processador durante a execução. Estes endereços são definidos de acordo com o espaço de endereçamento do processador.



Os processos “enxergam” somente os endereços lógicos (memória virtual)

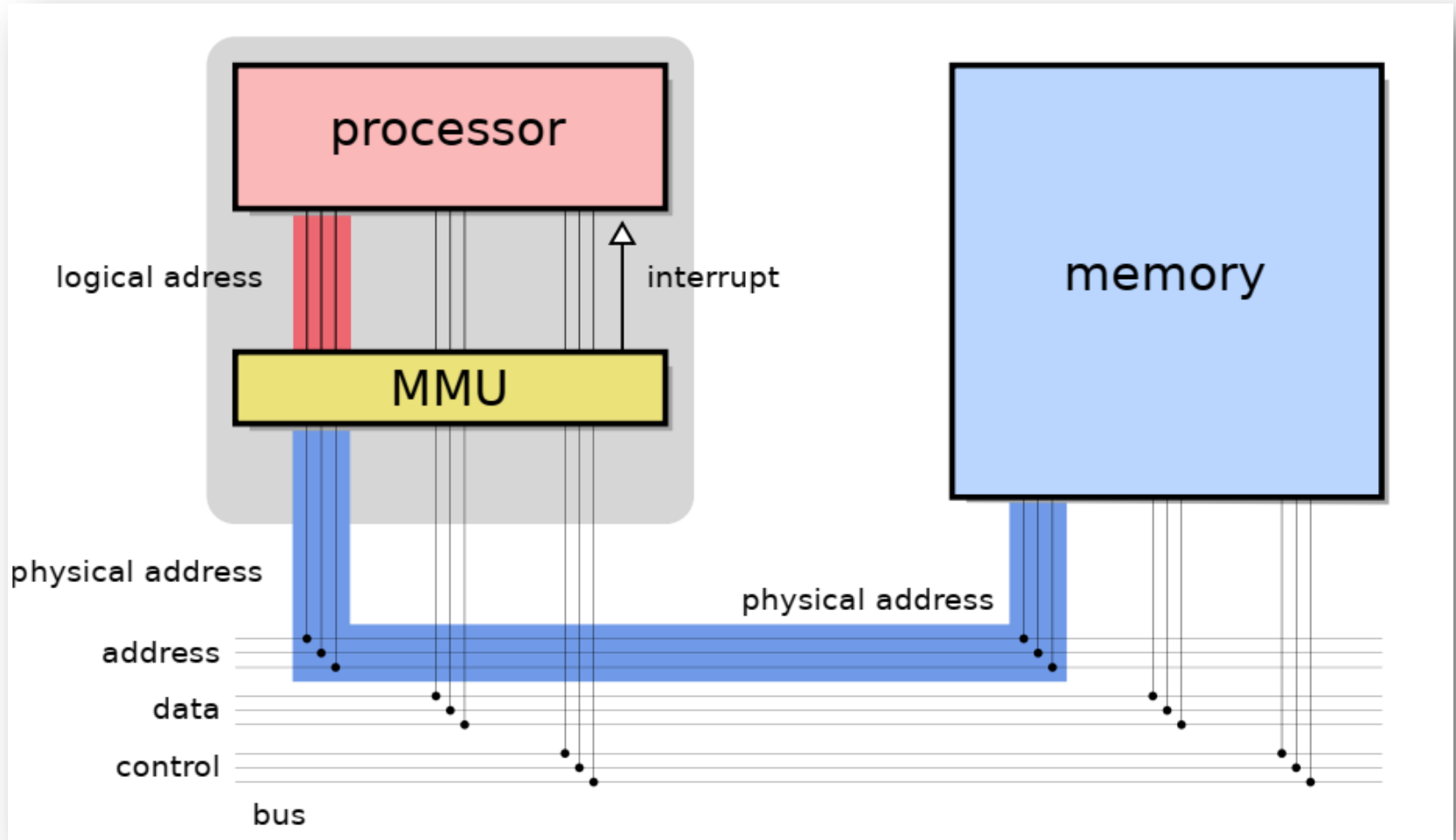
A memória Virtual

Por questões de desempenho, a tradução de endereços lógicos em físicos é feita por um **componente específico do hardware** do computador, denominado **Unidade de Gerência de Memória** (MMU – *Memory Management Unit*).

Na maioria dos processadores atuais, a MMU se encontra integrada ao chip da própria CPU.

- A MMU intercepta o acesso do processador ao barramento de endereços, recebendo os endereços lógicos gerados pelo mesmo e enviando os endereços físicos correspondentes.
- A MMU também tem acesso ao barramento de controle, para identificar operações de leitura e de escrita na memória.

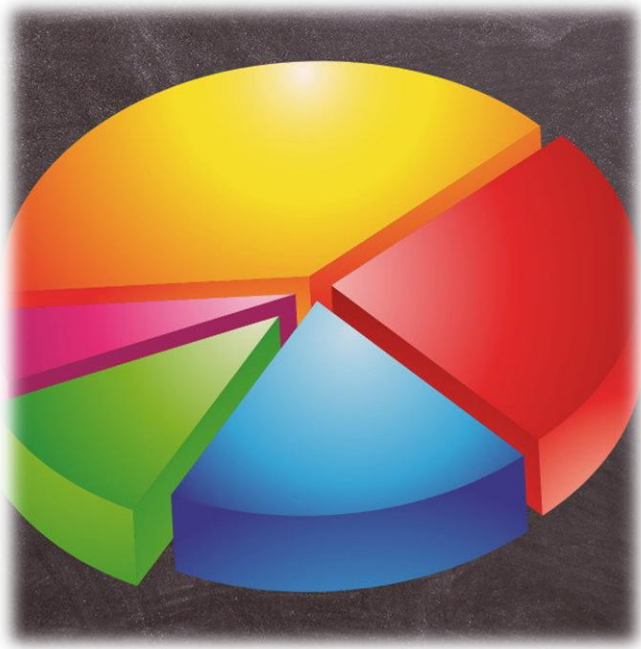
A memória Virtual



A memória Virtual

A implementação de Memória Virtual pode ser através de:

- Memória virtual por partições
- Memória virtual por segmentos
- Memória virtual por paginação



MEMÓRIA VIRTUAL POR PARTIÇÕES

Memória virtual por partições

Consiste em dividir a memória física em N partições (pedaços, blocos), que podem ter tamanhos iguais ou distintos, fixos ou variáveis.

Em cada partição da memória física é carregado um processo.

O processo ocupa uma partição de tamanho N bytes terá um espaço de endereçamento com até N bytes, com endereços lógicos no intervalo $[0 \dots N - 1]$.

Nessa implementação, a MMU possui dois registradores:

- um **registrador base** (B), que define o endereço físico inicial da partição,
- e um **registrador limite** (L), que define o tamanho em bytes dessa partição.

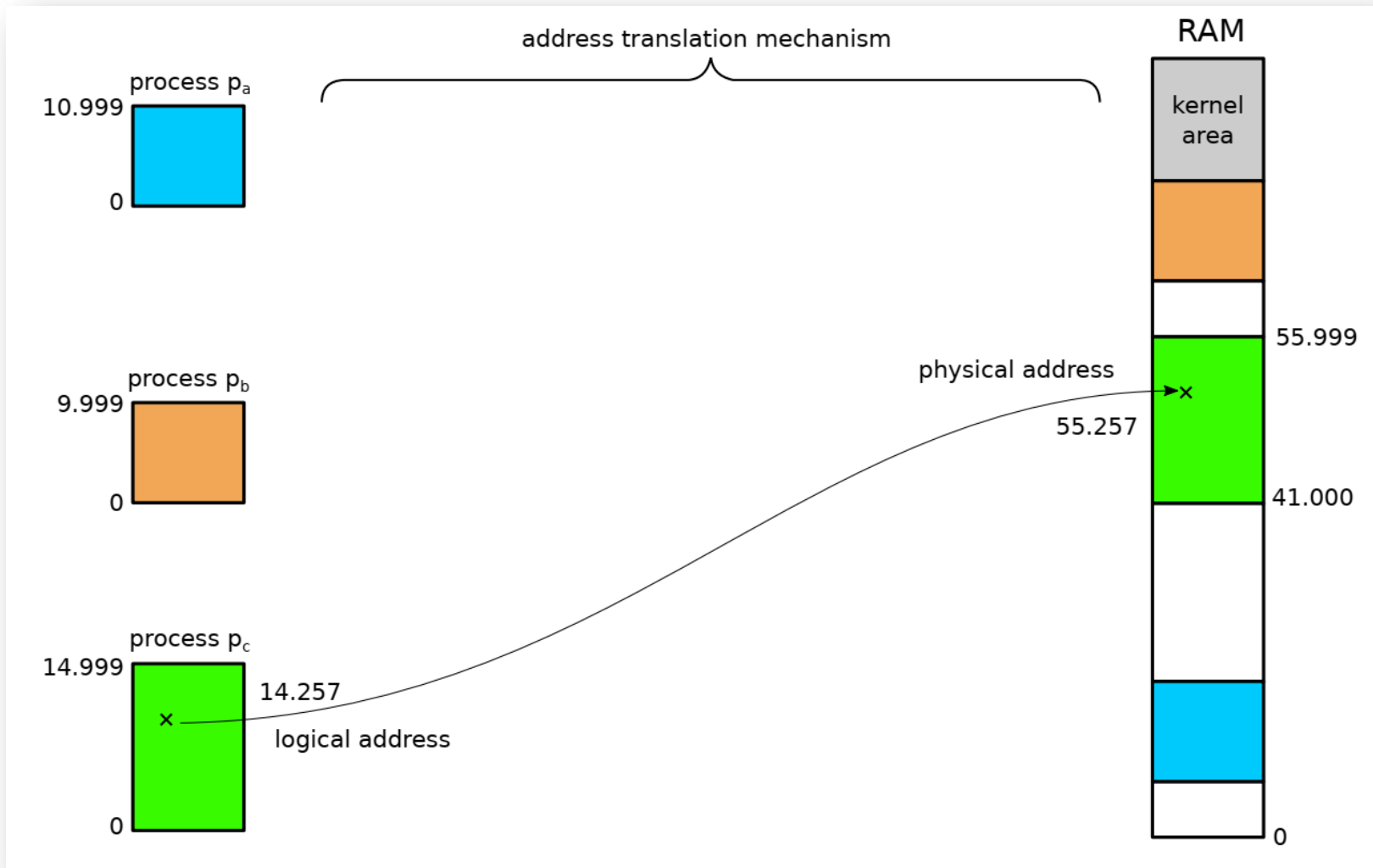
Memória virtual por partições

O Funcionamento é comparado e alterada para cada endereço lógico gerado (el) pelo processador, é comparado ao valor do registrador limite (L).

Caso seja menor que este ($el < L$), o endereço lógico é somado ao valor do registrador base, para a obtenção do endereço físico correspondente ($ef = el + B$).

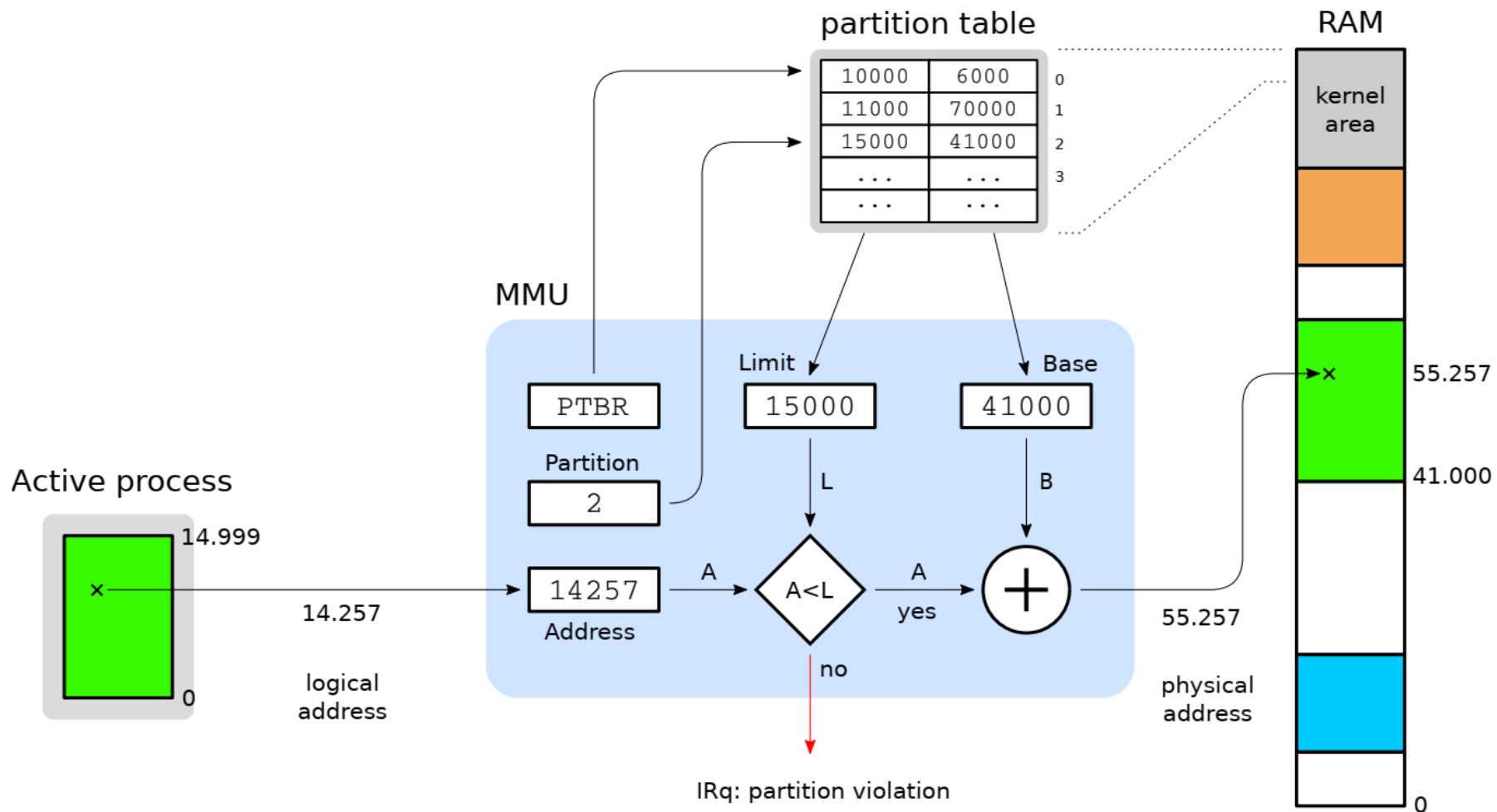
Caso contrário ($el \geq L$), uma interrupção é gerada pela MMU, indicando um endereço lógico inválido

Memória virtual por partições



O processo ativo, ocupando a partição 2, tenta acessar o endereço lógico 14.257. A MMU verifica que esse endereço é válido, pois é inferior ao limite da partição (15.000). Em seguida o endereço lógico é somado à base da partição (41.000) para obter o endereço físico correspondente (55.257).

Memória virtual por partições





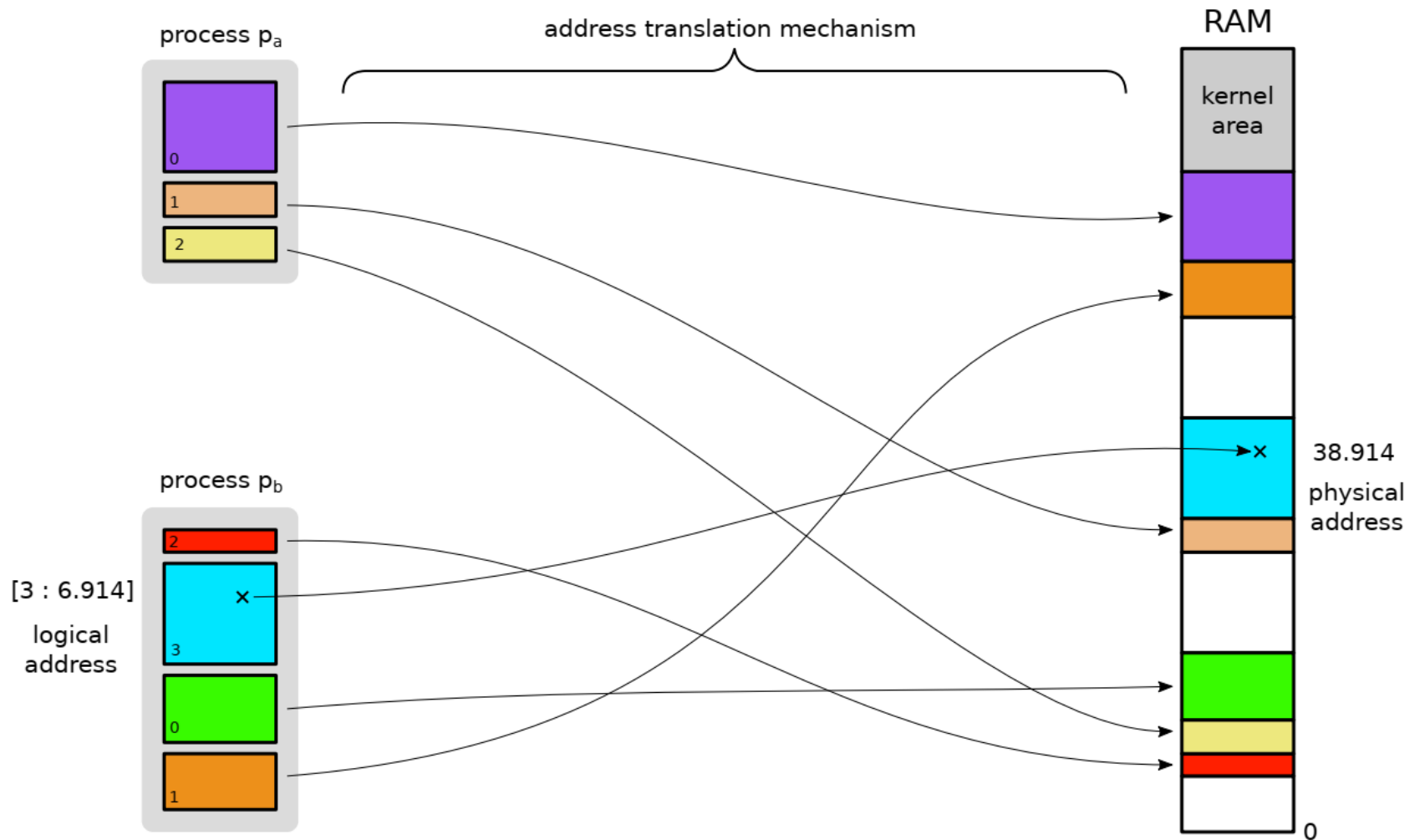
MEMÓRIA VIRTUAL POR SEGMENTOS

Memória virtual por Segmentos

Nesta abordagem, o espaço de endereçamento de cada processo não é mais visto como uma sequência linear de endereços lógicos, mas como uma coleção de áreas de tamanhos diversos e políticas de acesso distintas, **denominadas *segmentos***.

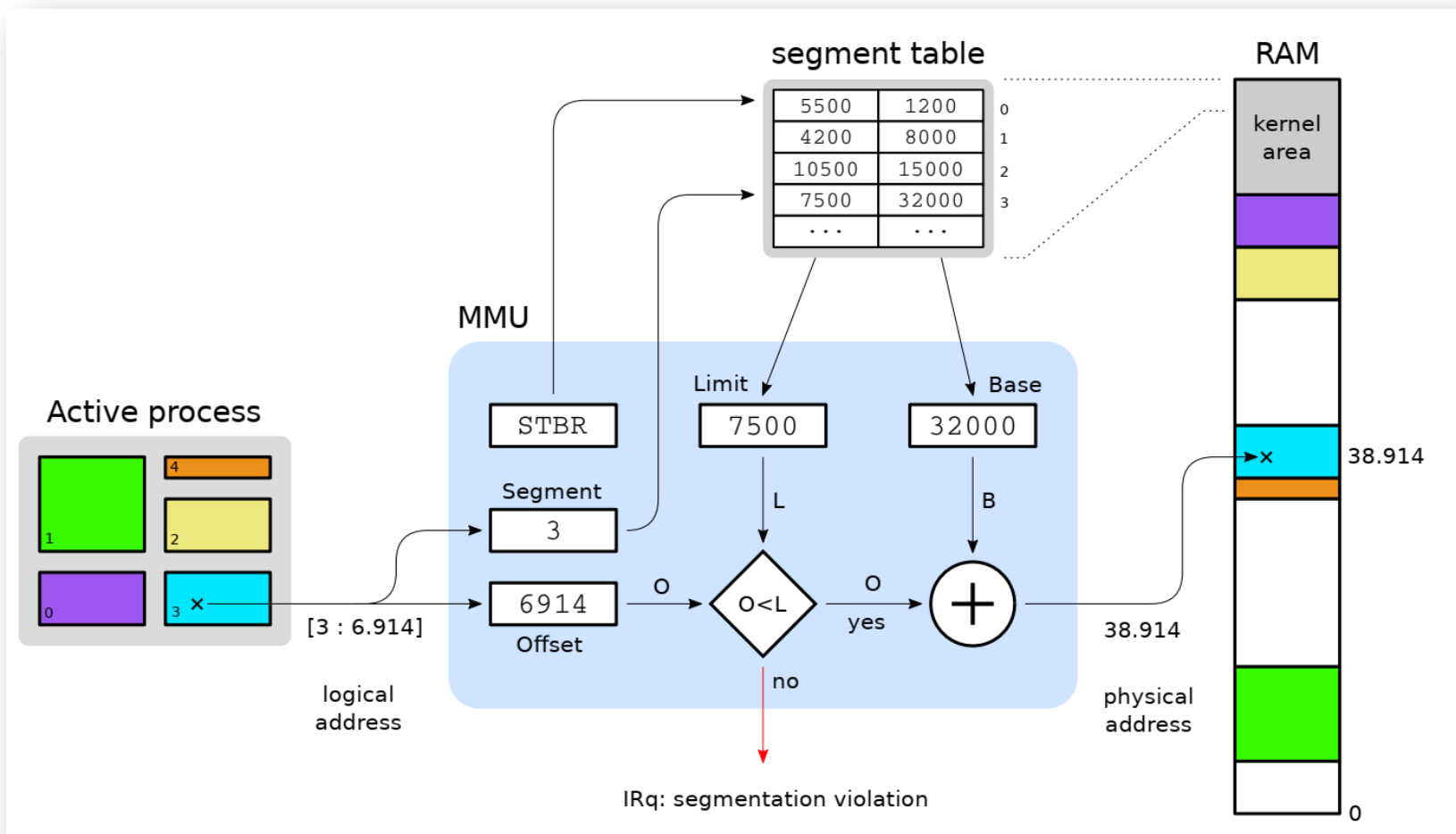
Os endereços lógicos gerados pelos processos são compostos por pares [***segmento*** : ***offset***], onde *segmento* indica o número do segmento e *offset* indica a posição dentro daquele segmento.

Memória virtual por Segmentos

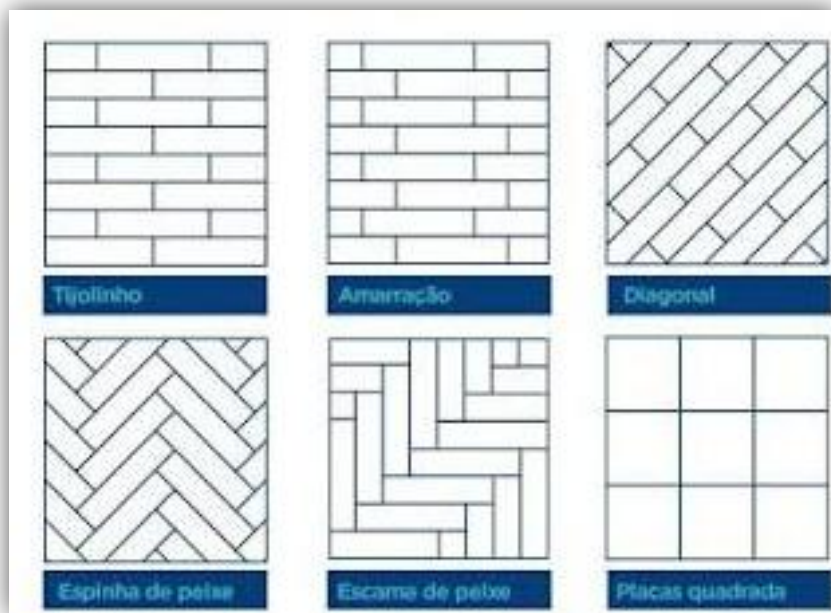


O endereço lógico [3 : 6.914], que corresponde ao *offset* 6.914 no segmento 3 do processo p_b . Nada impede de existir outros endereços 6.914 em outros segmentos do mesmo processo.

Memória virtual por Segmentos



O modelo de memória virtual por segmentos foi muito utilizado nos anos 1970-90, sobretudo nas arquiteturas Intel e AMD de 32 bits. Pode deixar lento devido ao alto custo de acesso as tabelas de segmentos armazenados na memória RAM.



MEMÓRIA VIRTUAL POR PAGINAÇÃO



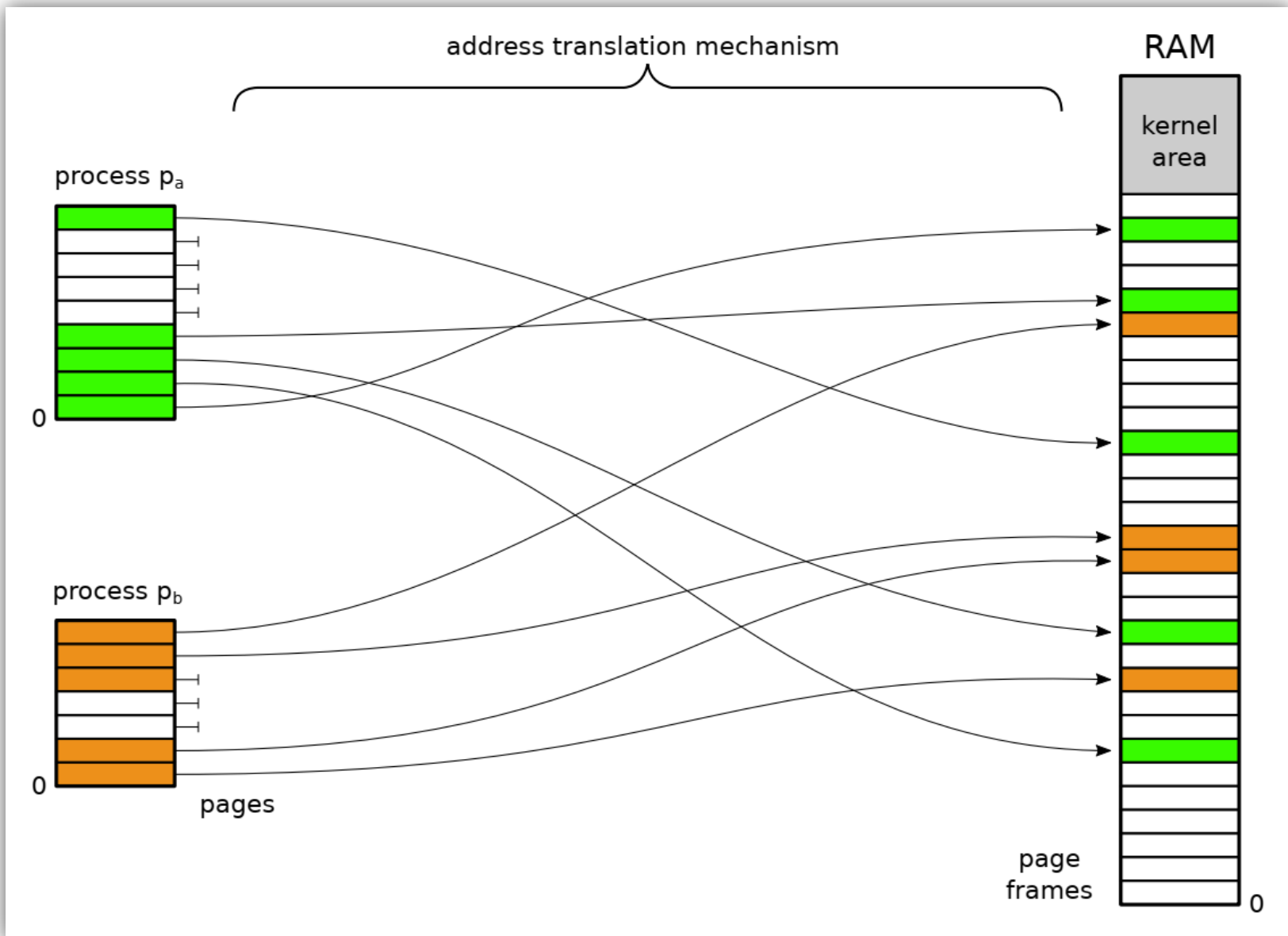
Memória virtual por Paginações

Nesse modelo, o espaço de endereçamento lógico dos processos é mantido linear e unidimensional.

Internamente, de forma transparente para o processador, o espaço de endereçamento lógico é dividido em pequenos blocos de mesmo tamanho, denominados *páginas*.

Nas arquiteturas atuais, as páginas geralmente têm 4 KBytes (4.096 bytes)

Memória virtual por Paginações



Memória virtual por Paginações

Um processador Intel 80386 usa endereços lógicos de 32 bits e páginas com 4 KBytes;

Um endereço lógico de 32 bits é decomposto em um *offset* de 12 bits, que representa uma posição entre 0 e 4.095 dentro da página, e um número de página com 20 bits.

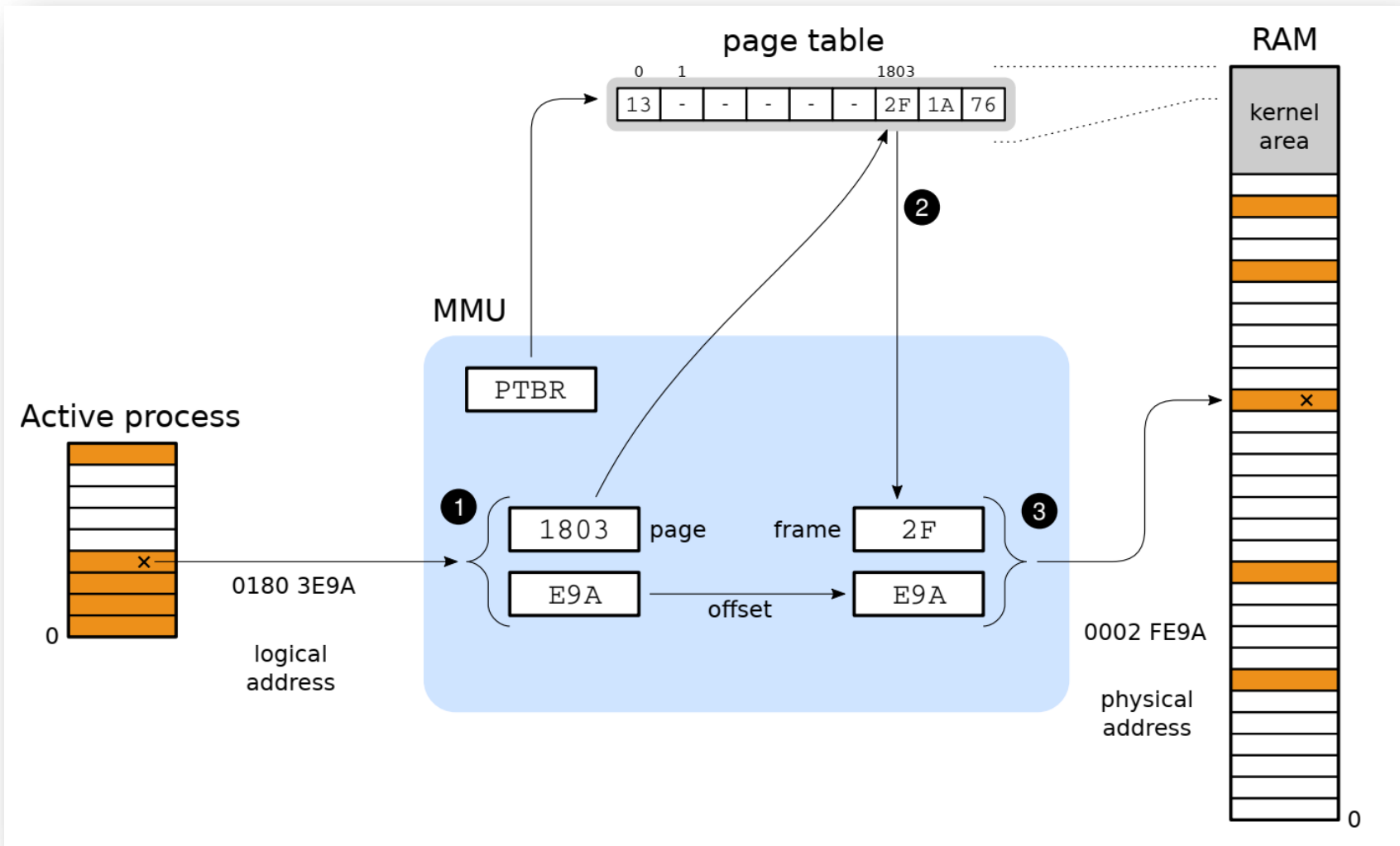
Dessa forma, podem ser endereçadas 2^{20} páginas com 2^{12} bytes cada (1.048.576 páginas com 4.096 bytes cada).

Memória virtual por Paginações

Eis um exemplo de decomposição do endereço lógico $01803E9A_h$ nesse sistema:

$$\begin{aligned} 01803E9A_h &\rightarrow \overbrace{0000\ 0001\ 1000\ 0000\ 0011}^{\text{page: 20 bits}} \overbrace{1110\ 1001\ 1010}_^{\text{offset: 12 bits}}_2 \\ &\rightarrow \overbrace{0000\ 0001\ 1000\ 0000\ 0011}^{\text{page}=01803_h} \overbrace{1110\ 1001\ 1010}_^{\text{offset}=E9A_h}_2 \\ &\rightarrow \text{page} = 01803_h \quad \text{offset} = E9A_h \end{aligned}$$

Memória virtual por Paginações





Memória virtual por paginações

Considerando que a tabela de páginas tem 2^{20} páginas, e cada entrada ocupa 4 bytes, uma tabela ocupará 4 MBytes de memória (4×2^{20} bytes) se for armazenada de forma linear na memória.

No caso de processos pequenos, com muitas páginas não mapeadas, uma tabela de páginas linear poderá ocupar mais espaço na memória que o próprio processo.

Para resolver esse problema, são usadas *tabelas de páginas multiníveis*, estruturadas na forma de árvores: uma primeira tabela de páginas (ou *diretório de páginas*) contém ponteiros para tabelas de páginas secundárias e assim por diante.

Quando uma tabela secundária não contiver entradas válidas, ela não precisa ser alocada; isso é representado por uma entrada nula na tabela principal.

Memória virtual por Paginações

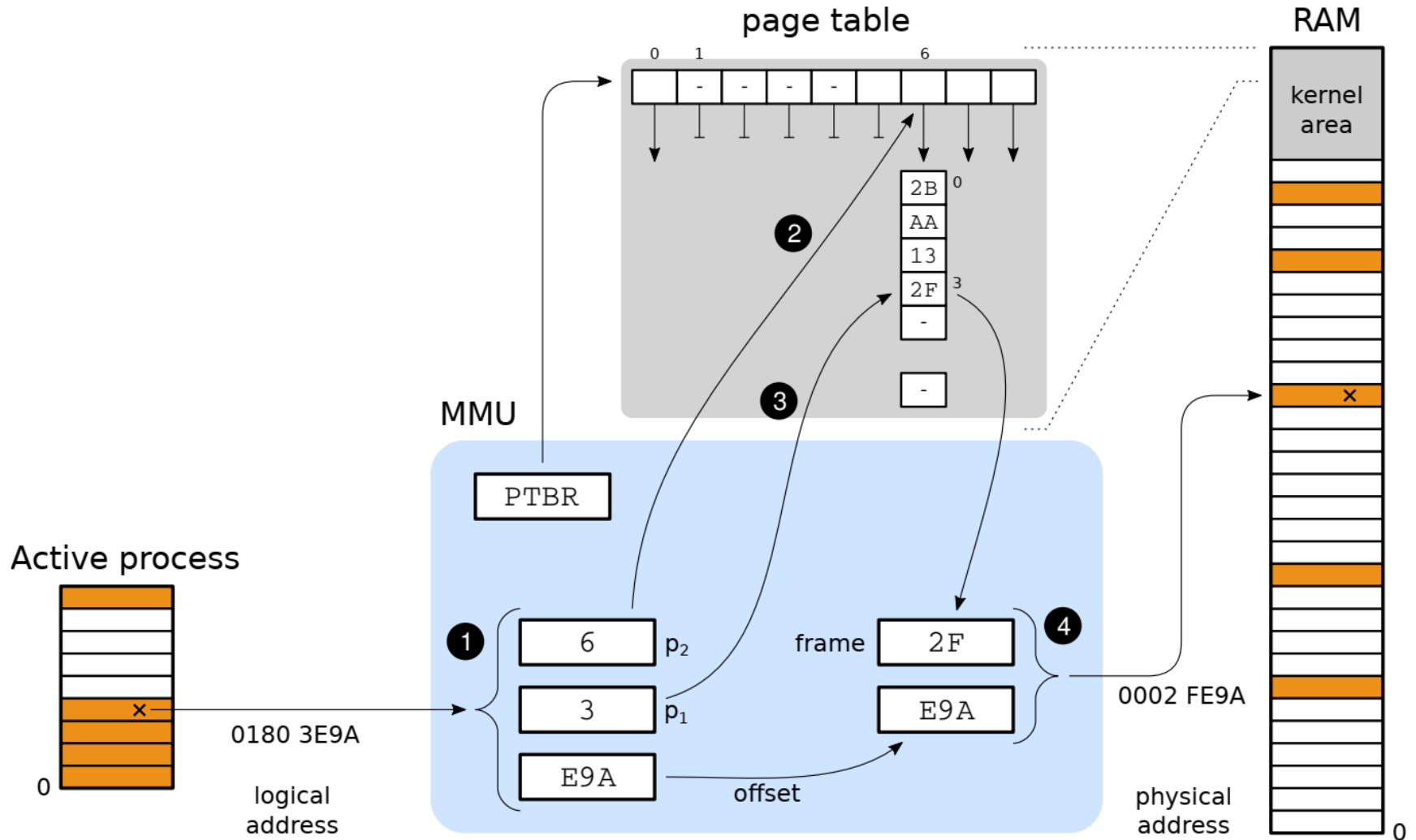
O número de página precisa ser dividido em duas ou mais partes, que são usadas como índices em cada nível de tabela, até encontrar o número de quadro desejado.

Considerando uma arquitetura de 32 bits com páginas de 4 KBytes, 20 bits são usados para acessar a tabela de páginas.

Esses 20 bits são divididos em dois grupos de 10 bits (p_1 e p_2) que são usados como índices em uma tabela de páginas com dois níveis:

$$\begin{aligned} 01803E9A_h &\rightarrow \overbrace{0000\ 0001}^{p_2:10\ bits} \overbrace{10\ 00\ 0000\ 0011}^{p_1:10\ bits} \overbrace{1110\ 1001\ 1010}^{offset:12bits} \\ &\rightarrow \overbrace{0000\ 0001}^{p_2=0006_h} \overbrace{10\ 00\ 0000\ 0011}^{p_1=0003_h} \overbrace{1110\ 1001\ 1010}^{offset=E9A_h} \\ &\rightarrow p_2 = 0006_h \quad p_1 = 0003_h \quad offset = E9A_h \end{aligned}$$

Memória virtual por Paginações

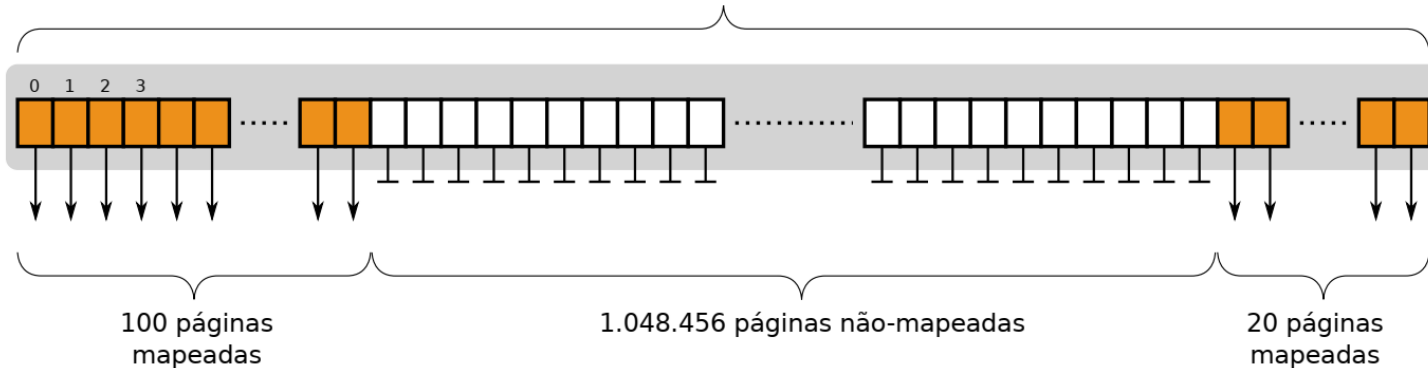




INSTITUTO FEDERAL
RIO GRANDE DO SUL

Memória virtual por Paginações

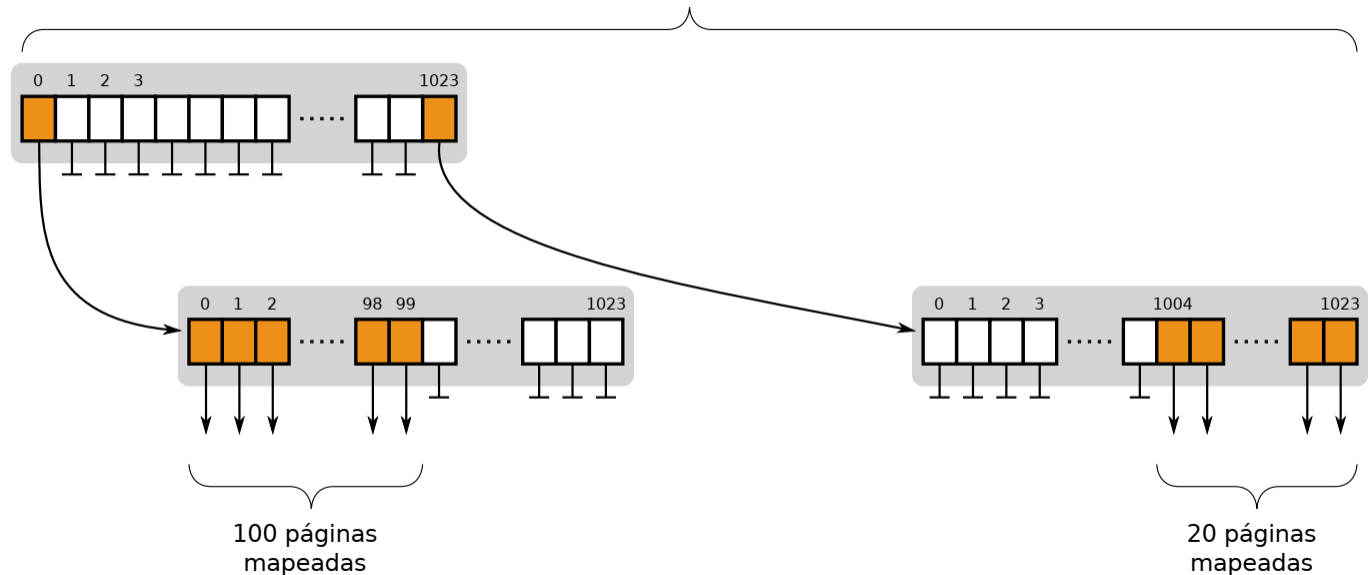
1.048.576 ponteiros de 32 bits = 4 MBytes



Modo Linear x Dois Níveis

Processo com
120
páginas em
RAM, ou 480
KBytes .

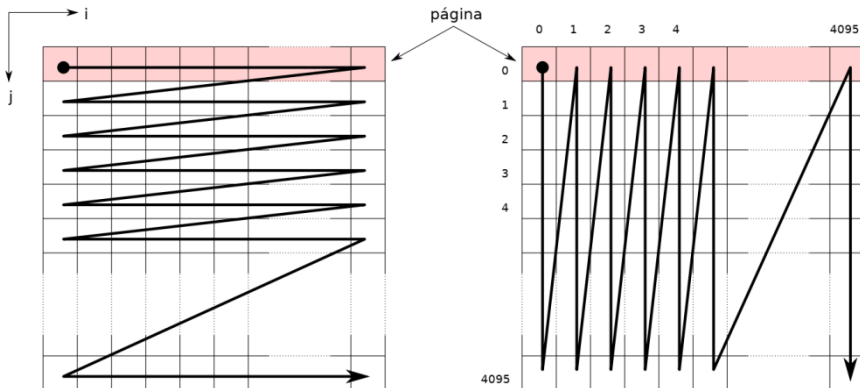
3 x 1.024 ponteiros de 32 bits = 12 KBytes





Memória virtual por Paginações

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 unsigned char buffer[4096][4096] ;
5
6 int main ()
7 {
8     int i, j ;
9
10    for (i=0; i<4096; i++)
11        for (j=0; j<4096; j++)
12            buffer[i][j]= (i+j) % 256 ;
13 }
```



Qual é o mais rápido?

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 unsigned char buffer[4096][4096] ;
5
6 int main ()
7 {
8     int i, j ;
9
10    for (i=0; i<4096; i++)
11        for (j=0; j<4096; j++)
12            buffer[j][i]= (i+j) % 256 ;
13 }
```



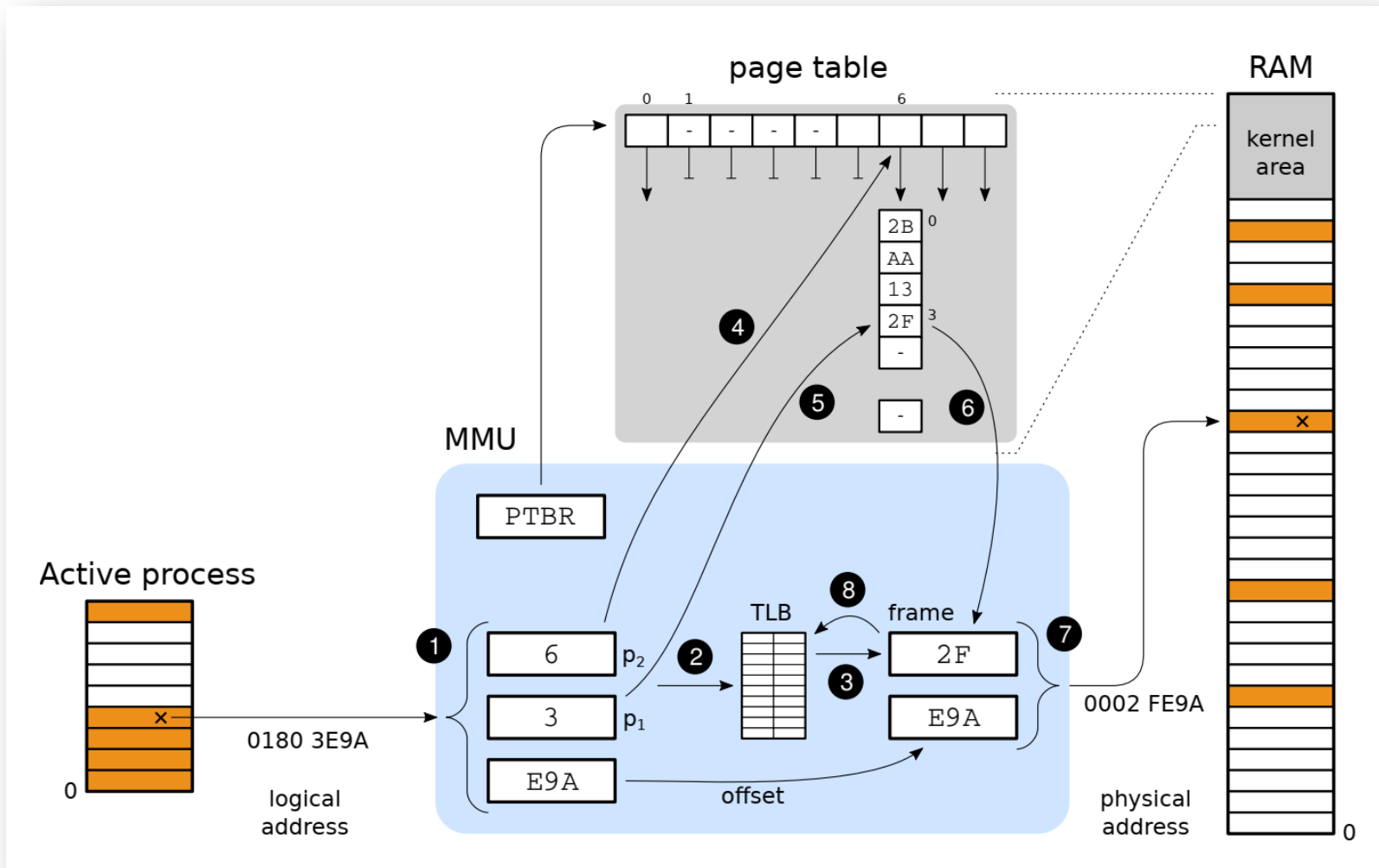
Memória virtual por Paginações

Cache da tabela de páginas

A estruturação das tabelas de páginas em vários níveis resolve o problema do espaço ocupado pelas tabelas, mas tem um efeito colateral nocivo: **aumenta fortemente o tempo de acesso à memória.**

Como as tabelas de páginas são armazenadas na memória RAM, cada acesso a um endereço de memória implica em mais acessos para percorrer a árvore de tabelas e encontrar o número de quadro desejado.

Memória virtual por Paginações



O cache de tabela de páginas na MMU, denominado **TLB (*Translation Lookaside Buffer*)** armazena as consultas recentes às tabelas de páginas do processo ativo.